工业 AI 质检与时序分析平台

--设计说明书

第一章　前言

1. 系统概述

工业 AI 质检与时序分析平台面向制造场景的图像缺陷检测与时序异常分析，提供从数据接入、预处理、索引到在线服务的一体化能力。后端基于 FastAPI + Celery + Redis，支持图像缺陷样本管理、检测任务触发、结果查询与导出，以及时序数据集管理、异常检测与报告；前端基于 React + Ant Design 提供仪表盘、任务与设备管理、通知中心等界面。平台通过角色权限控制与审计日志保障安全，并可在容器化环境下快速部署。

2. 开发环境

软件栈：

Python 3.11

FastAPI / Uvicorn / SQLAlchemy (async) / Alembic

Redis 7 作为缓存与 Celery 队列

PostgreSQL 15 作为主数据库

前端：Node.js (18+ 推荐) + Vite + React 18 + Ant Design 5 + ECharts

软件部署：

docker，docker compose

开发语言：

Python, TypeScript / JavaScript

3. 运行环境

硬件建议（服务器）：CPU ≥ 8 核，内存 ≥ 32 – 64 GB，磁盘 ≥ 1 – 2 TB（含存储卷）

软件（服务器）：Ubuntu 22.04/24.04，Docker 20+，Docker Compose 2+

客户端：现代浏览器（Edge / Chrome），1080p 及以上分辨率

4. 软件故障排除

查看服务状态：

docker ps | findstr industrial-ai（或在 Linux 用 grep）

服务管理（项目根目录）：

启动：docker compose up -d

停止：docker compose down

核心容器日志：

后端 API：docker logs -f industrial-ai-backend

前端：docker logs -f industrial-ai-frontend

Celery Worker：docker logs -f industrial-ai-celery-worker

Celery Beat：docker logs -f industrial-ai-celery-beat

Redis：docker logs -f industrial-ai-redis

PostgreSQL：docker logs -f industrial-ai-postgres

健康检查：后端提供 http://localhost:8000/api/v1/health（Compose 中已作为 healthcheck）

常见排查要点：确认 env 中 DATABASE_URL、REDIS_URL 配置正确；确认存储卷 ./storage 可写；查看 Alembic 迁移是否完成；若端口被占用，调整 Compose 映射。

第二章　总体设计

1. 功能结构图

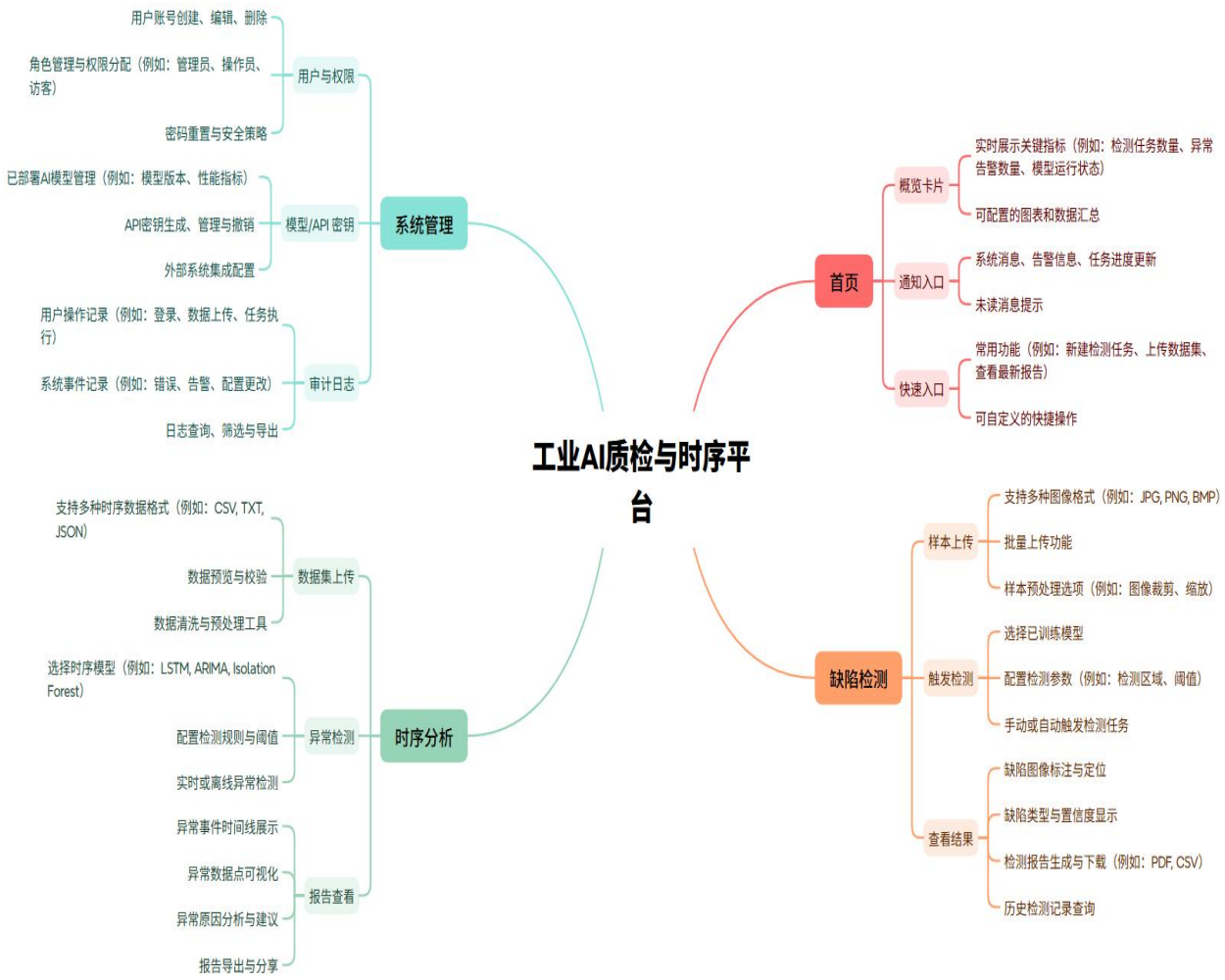


图 1 功能结构图

2. 界面效果图

2.1 首页

平台为本地部署模式，部署完成后，通过在浏览器输入系统地址，进入系统首页。

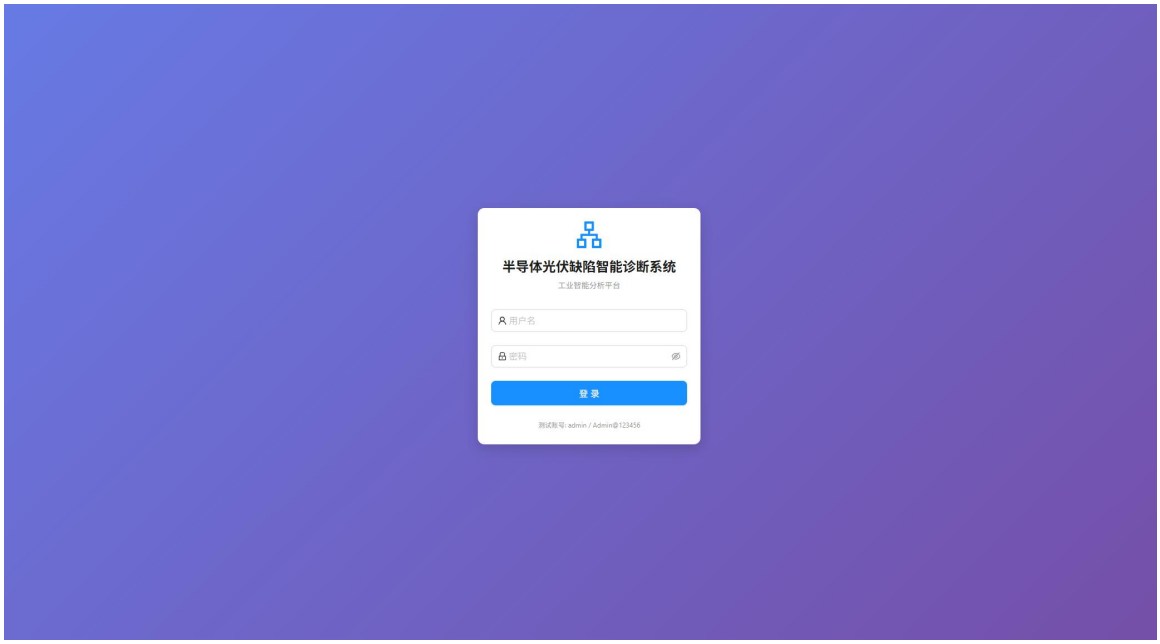


图 2 登录界面

该界面为半导体光伏缺陷智能诊断系统的登录页面，用于实现用户身份认证与系统访问控制。页面整体采用渐变色背景设计，中央为登录功能模块，结构简洁清晰。

2.2 系统功能

本系统围绕半导体光伏生产过程中的设备运行监测与缺陷智能诊断需求进行设计与实现，构建了集数据采集、设备管理、产线管理、样本管理、时序数据分析与智能诊断于一体的综合管理平台。系统支持设备基础信息维护与运行状态实时展示，通过健康评分与状态标签实现设备运行状况的可视化监控；支持缺陷样本上传与管理，为模型训练与优化提供数据支撑；提供时序数据集管理与模拟数据生成能力，满足算法测试与性能验

证需求；同时通过产线与设备的关联管理，实现生产结构的数字化建模与统一调度。系统采用模块化架构设计，各功能模块相互协同，在保障数据安全与权限控制的前提下，实现对生产设备运行状态的智能分析与辅助决策支持，从而提升生产效率与运维管理水平

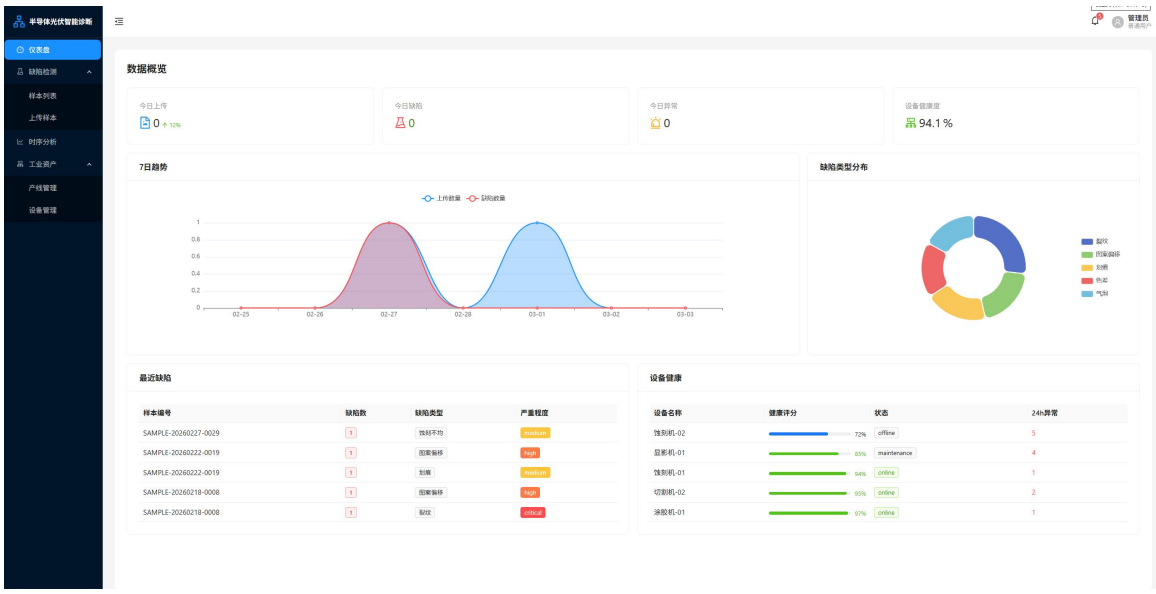


图 3 设备管理界面

设备管理模块用于统一管理系统中的所有生产设备信息，实现设备的新增、查询、编辑与删除等操作。界面顶部提供关键字搜索功能，可根据设备编号或设备名称进行快速检索。列表区域展示设备编号、设备名称、设备类型、运行状态、健康评分、所属产线、位置及最后数据时间等信息。其中运行状态采用标签形式标识（如“在线”“离线”“维护中”），健康评分以进度条形式直观展示设备当前运行状况。用户可通过右侧操作栏进行编辑或删除操作，实现设备信息的动态维护与更新。

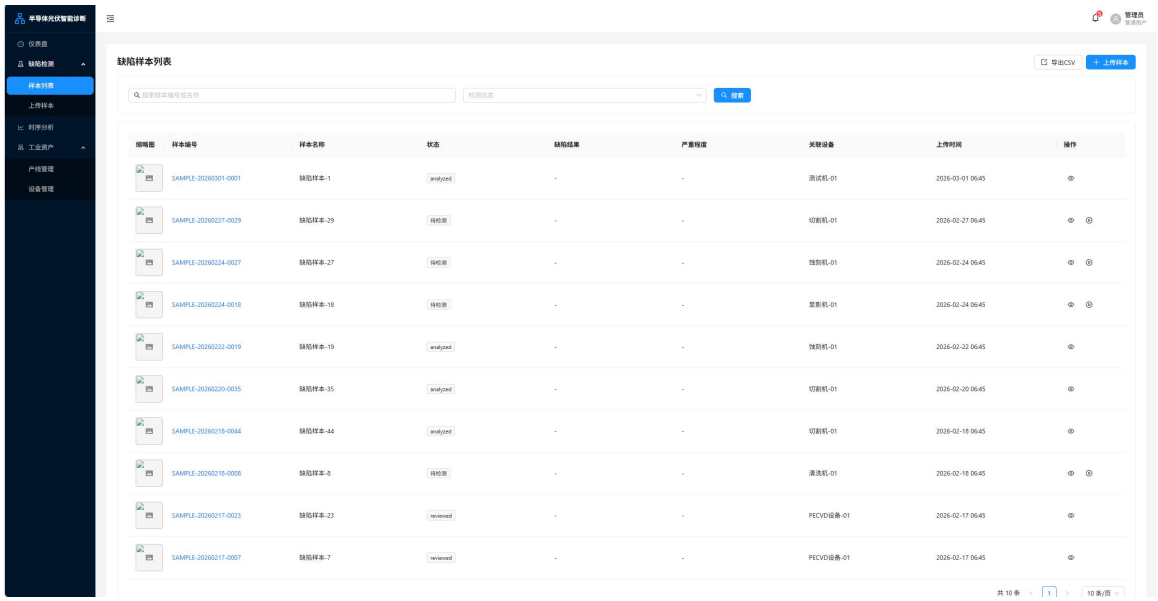


图 4 上传缺陷样本

点击“新增设备”按钮后，系统弹出新增设备表单窗口。用户需填写设备基础信息，包括设备编号、设备名称、设备类型、设备型号、制造商、设备位置及设备描述等内容。

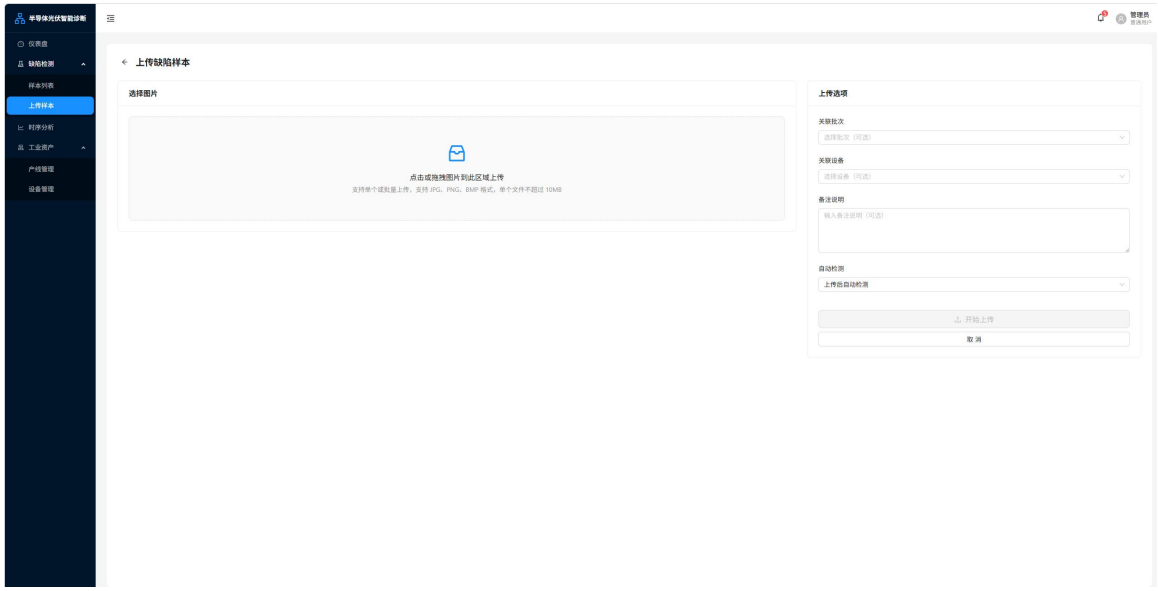


图 5 时序数据集列表

删除操作：对不再使用的知识库，可在知识库详情页面中点击“删除知识库”按钮，系统会弹出确认提示。确认后，该知识库会被标记为删除状态并释放相应配额，同时通过后台任务逐步清理相关索引和资源。

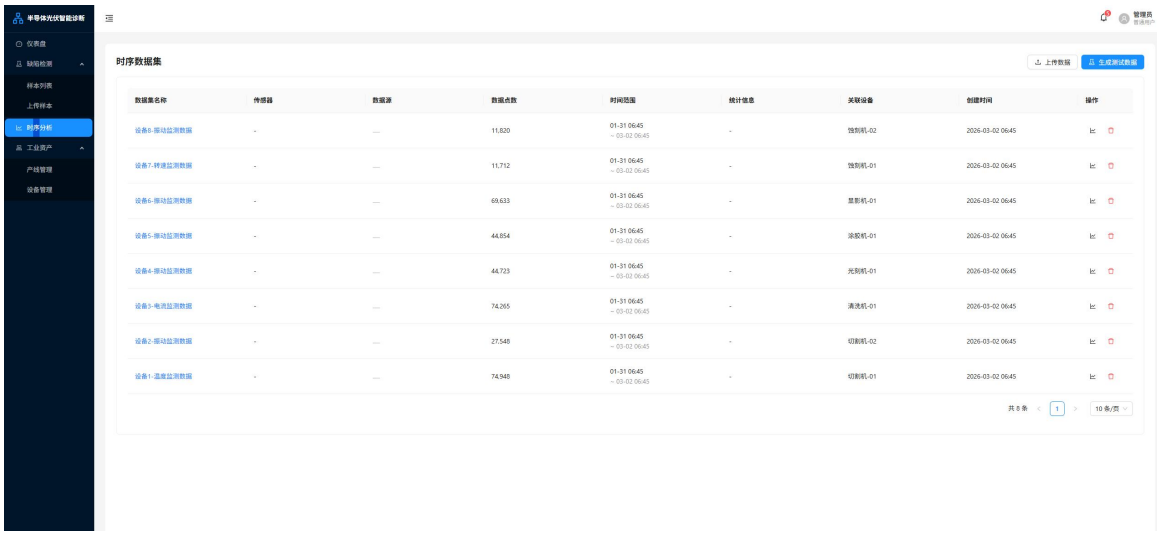


图 6 生成模拟数据集

文档管理与索引：在知识库详情页中，可为该知识库上传多种格式的文档。系统支持批量文档上传与单文档上传，自动完成文件类型与大小校验、内容哈希去重，将文件存入对象存储并记录元数据。根据知识库配置，文档会被解析、分片，并构建向量索引、全文索引、图索引、摘要索引和视觉索引等。文档列表支持分页、排序和按名称搜索，并提供预览原文、单文档或批量删除、针对指定文档或知识库内失败索引的重建等操作。

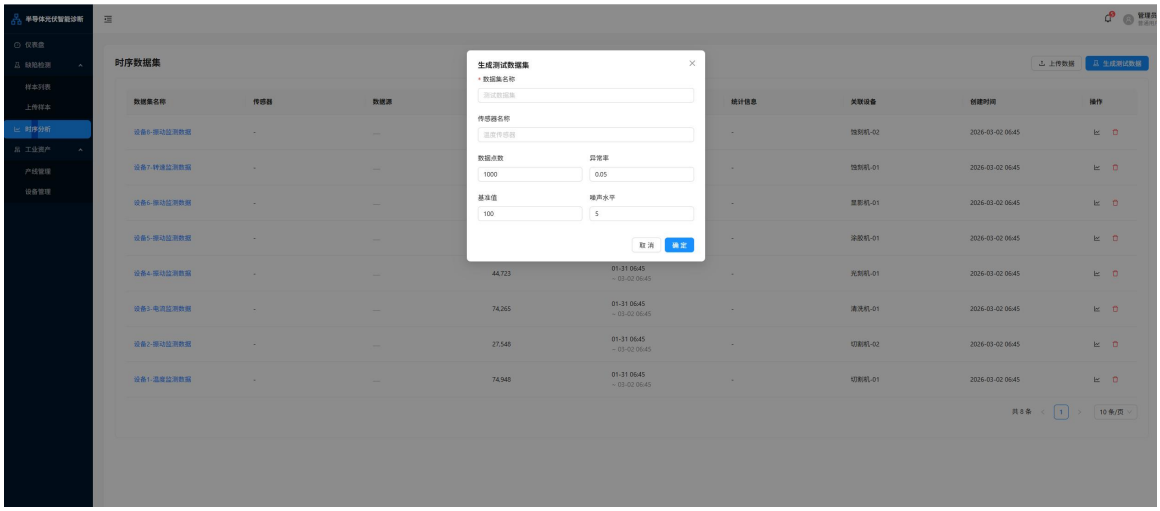


图 7 产线管理界面

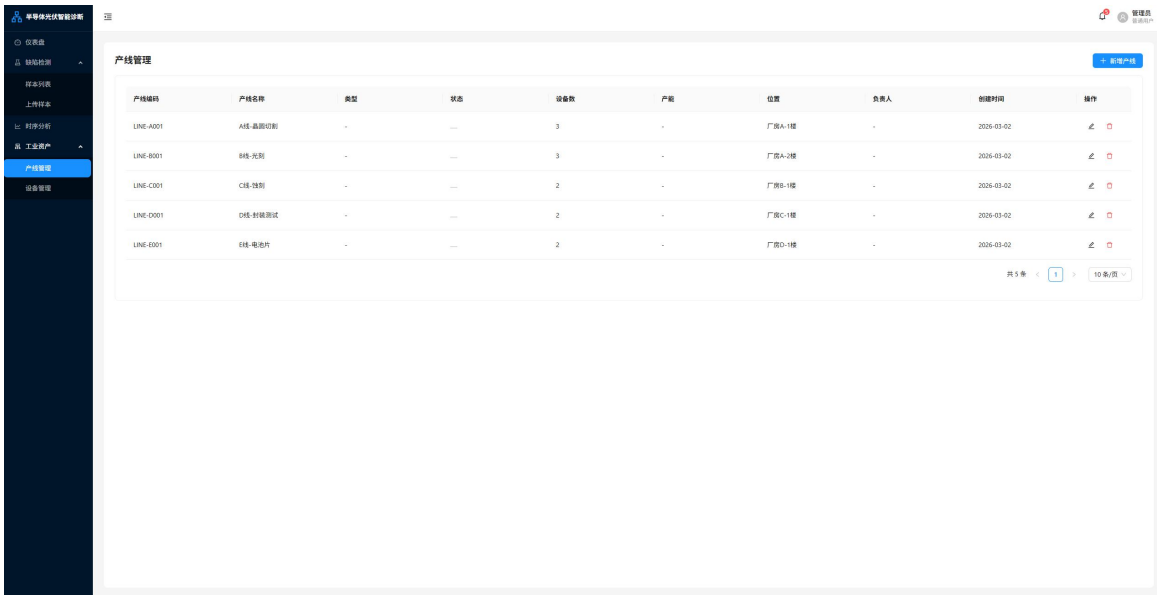


图 8 新增产线

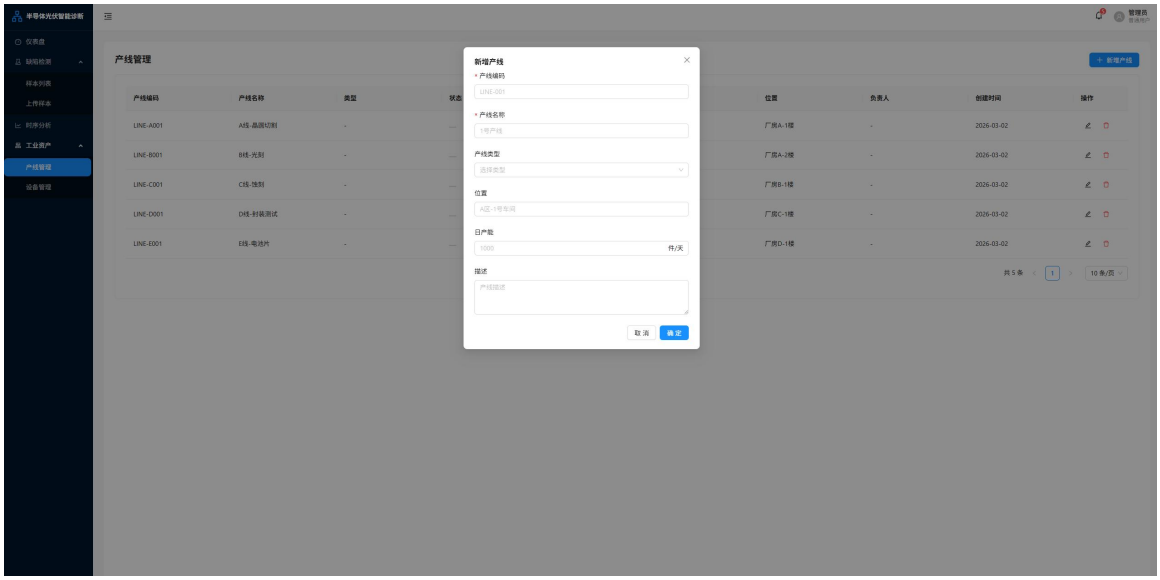


图 9 设备管理列表（完整视图）

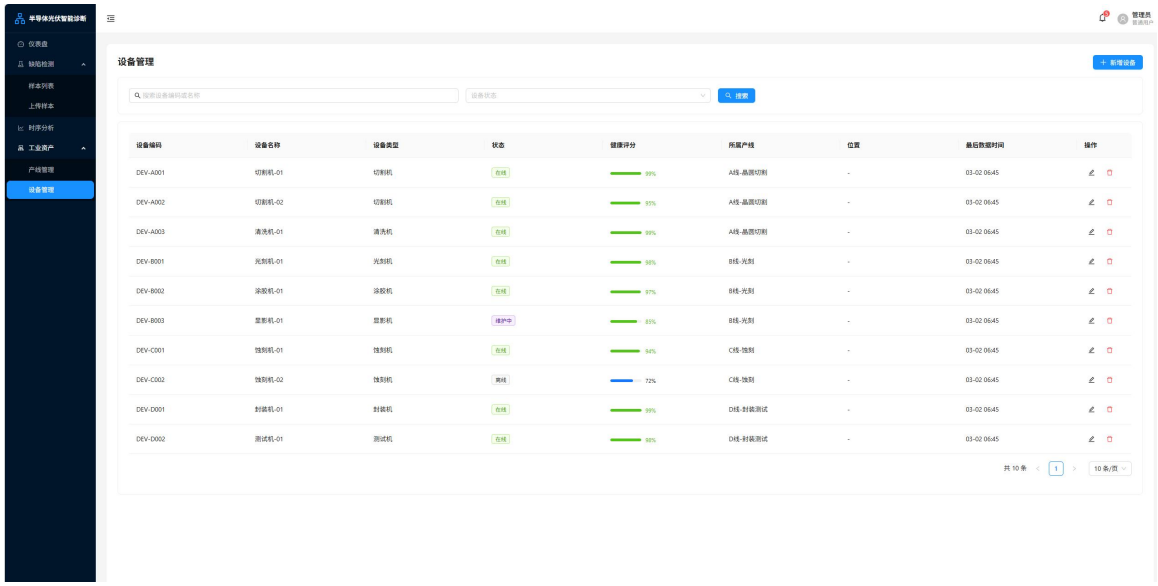


图 10 设备管理主界面

该界面为设备管理模块的主页面，用于集中展示系统中所有设备的运行信息与状态数据。页面顶部提供设备编号或设备名称的搜索框，支持模糊查询；右上角设置“新增设备”按钮，用于添加新的设备信息。

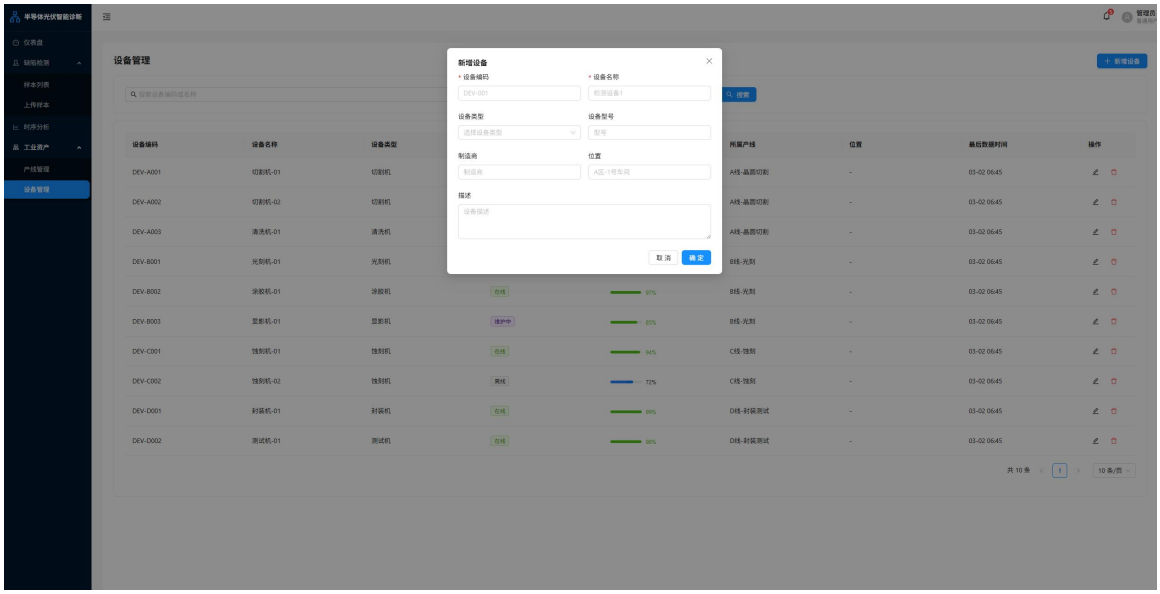


图 11 新增设备弹窗界面

在设备管理主界面点击“新增设备”按钮后，系统弹出新增设备表单窗口，用于录入设备基础信息。

第三章 接口设计

1. 认证

1.1 登录

请求路径：/api/v1/auth/login

请求方式：POST

请求参数：

```
{
  "username": "admin",
  "password": "Admin@123456"
}
```

返回格式：

```
{
  "access_token": "eyJhbGciOi...",
  "refresh_token": "eyJhbGciOi...",
  "token_type": "bearer",
  "expires_in": 1800
}
```

2. 缺陷检测

2.1 上传缺陷样本

请求路径：/api/v1/defect/samples/upload

请求方式：POST (multipart/form-data)

返回格式：

```
file: <图片文件>
sample_no: "DS20260304120001"
name: "侧板划伤"
device_id: "<设备 UUID>"
batch_id: "<批次 UUID>"
```

2.2 会话列表

请求路径：/api/v1/bots/{bot_id}/chats

请求方式：GET

请求参数：page、page_size

返回格式：


```
{
  "items": [
    {
      "id": "chat_1001",
      "title": "产品咨询",
      "bot_id": "bot_001",
      "status": "active",
      "created": "2025-12-30T09:00:00.000000Z",
      "updated": "2025-12-30T09:10:00.000000Z"
    }
  ],
  "page": 1,
  "page_size": 50,
  "total": 1,
  "total_pages": 1
}
```


2.3 会话详情

请求路径：/api/v1/bots/{bot_id}/chats/{chat_id}

请求方式：GET

返回格式：


```
{
  "id": "chat_1001",
  "title": "产品咨询",
  "bot_id": "bot_001",
  "peer_type": "web",
  "history": [
    [
      {
```

```
            "id": "m_1",
            "type": "message",
            "role": "human",
            "timestamp": 1735549200.0,
            "data": "这款产品支持哪些操作系统？",
            "files": []
          },
          {
            "id": "m_2",
            "type": "message",
            "role": "ai",
            "timestamp": 1735549202.5,
            "data": "当前版本支持 Windows 10 及以上，macOS 12 及以上。",
            "references": [],
            "urls": []
          }
        ]
      ],
      "status": "active",
      "created": "2025-12-30T09:00:00.000000Z",
      "updated": "2025-12-30T09:10:00.000000Z"
    }
```


2.4 触发缺陷检测

请求路径：/api/v1/defect/detect

请求方式：POST

请求参数：

```
{
  "sample_ids": [
    "f9c9c2c0-...-9a77",
    "2c3c6c10-...-b8a2"
  ],
  "detector": "default"      # 可选模型标识
}
```

2.5 获取样本详情

请求路径：/api/v1/chat/completions/frontend/api/v1/defect/samples/{sample_id}

请求方式：POST

请求参数:stream、bot_id、chat_id

返回格式：

```
{
  "data": {
    "id": "f9c9c2c0-...-9a77",
    "sample_no": "DS20260304120001",
    "result": {
      "defect_type": "scratch",
      "confidence": 0.93,
      "bbox": [120, 80, 240, 190]
    },
    "status": "completed",
    "created_at": "2026-03-04T04:30:10Z"
  }
}
```

2.6 删除会话

请求路径：/api/v1/bots/{bot_id}/chats/{chat_id}

请求方式：DELETE

返回格式：

204，无body

3. 时序分析

3.1 上传时序数据集

3.1.1 获取某Provider的模型列表

请求路径：/api/v1/timeseries/datasets/upload

请求方式：GETPOST (multipart/form-data)

返回格式：

```
{

  "data": {

    "id": "7e1a...c4",

    "name": "产线振动监测-3 月",

    "points_count": 12000,

    "status": "ready"

  }

}
```


3.1.2 新增模型

请求路径：/api/v1/llm_providers/{provider_name}/models

请求方式：POST

请求参数：


```
{
  "provider_name": "openai",
  "model": "gpt-4.1-large",
  "api": "completion",
  "custom_llm_provider": "openai",
  "context_window": 128000,
  "max_input_tokens": 8000,
  "max_output_tokens": 4000
}
```


返回格式：


```
{
  "provider_name": "openai",
  "model": "gpt-4.1-large",
```

```
    "api": "completion",
    "custom_llm_provider": "openai",
    "context_window": 128000,
    "max_input_tokens": 8000,
    "max_output_tokens": 4000,
    "tags": []
  }
```

3.1.3 编辑模型

请求路径：/api/v1/llm_providers/{provider_name}/models/{api}/{model}

请求方式：PUT

请求参数：

```
  {
    "custom_llm_provider": "openai",
    "context_window": 128000,
    "max_input_tokens": 120000,
    "max_output_tokens": 8000,
    "tags": ["free", "recommend"]
  }
```

返回格式：

```
  {
    "provider_name": "openai",
    "api": "completion",
    "model": "gpt-4o-mini",
    "custom_llm_provider": "openai",
    "context_window": 128000,
    "max_input_tokens": 120000,
    "max_output_tokens": 8000,
    "tags": ["free", "recommend"],
    "created": "2025-01-01T10:00:00Z",
    "updated": "2025-01-02T08:00:00Z"
  }
```

3.1.4 删除模型

请求路径：/api/v1/llm_providers/{provider_name}/models/{api}/{model}

请求方式：DELETE

请求参数：

provider_name：Provider 名

api：completion / embedding / rerank

model：模型名（URL 编码后放入路径）

返回格式：

成功：200 OK

失败：401：未授权

404：模型不存在

3.2 API 秘钥管理

3.2.1 获取当前用户 API Key 列表

请求路径：/api/v1/apikeys

请求方式：GET

返回格式：

```
{
  "items": [
    {
      "id": "k_1001",
      "key": "sk-xxxx",        // 通常前端只展示部分
      "description": "内部集成用 Key",
      "is_system": false,
      "created": "2025-12-30T10:00:00.000000Z",
      "updated": "2025-12-30T10:00:00.000000Z"
    }
  ]
}
```

3.2.2 创建API Key

请求路径：/api/v1/apikeys

请求方式：POST

请求参数：

```
{
  "description": "用于 CI 测试调用",
  "is_system": false
}
```

返回格式：

```
{
  "id": "k_1002",
  "key": "sk-new-key-xxxxx",
  "description": "用于 CI 测试调用",
  "is_system": false,
  "created": "2025-12-30T10:10:00.000000Z",
  "updated": "2025-12-30T10:10:00.000000Z"
}
```

3.2.3 更新API Key

请求路径：/api/v1/apikeys/{apikey_id}

请求方式：PUT

请求参数：

```
{
  "description": "用于前端集成测试的 Key"
}
```

返回格式：

```
{
  "id": "ak_123456",
  "key": "sk-xxxxxxxxxxxxxxxx",
  "description": "用于前端集成测试的 Key",
  "created_at": "2025-01-01T10:00:00Z",
  "updated_at": "2025-01-02T09:00:00Z",
  "last_used_at": "2025-01-02T08:30:00Z"
```

}

3.2.4 删除 API Key

请求路径：/api/v1/apikeys/{apikey_id}

请求方式：DELETE

请求参数：

apikey_id：要删除的 API Key ID

返回格式：

成功：200 OK，通常无复杂正文（OpenAPI 仅描述为“API key deleted successfully”，前端一般只根据状态码与无异常判断成功）

失败：401 未授权：返回 failResponse，表示当前用户未登录或无权限

404 未找到：返回 failResponse，表示该 apikey_id 不存在或不属于当前用户

4. 通知与健康检查

4.1 通知列表

请求路径：/api/v1/notifications

请求方式：GET

查询参数：page、page_size、is_read、notification_type

返回格式：

```
{
  "data": [
    { "id": "n1", "title": "检测完成", "is_read": false, "created_at": "2026-03-04T04:35:00Z" }
  ],
  "meta": { "page": 1, "page_size": 20, "total": 3 }
}
```

4.2 健康检查

请求路径：/api/v1/notifications

请求方式：GET

查询参数：page、page_size、is_read、notification_type

返回格式：


```
{
  "data": [
    { "id": "n1", "title": "检测完成", "is_read": false, "created_at": "2026-03-04T04:35:00Z" }
  ],
  "meta": { "page": 1, "page_size": 20, "total": 3 }
```


4.3 健康检查

请求路径：/api/v1/notifications

请求方式：GET

查询参数：page、page_size、is_read、notification_type

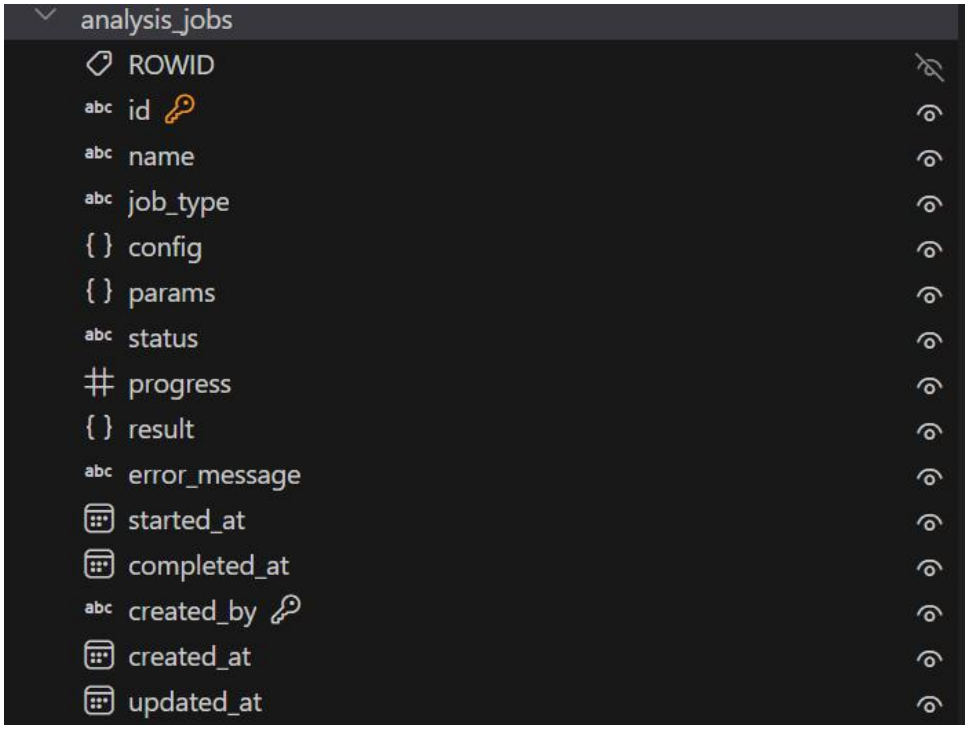
返回格式：


```
{
  "data": [
    { "id": "n1", "title": "检测完成", "is_read": false, "created_at": "2026-03-04T04:35:00Z" }
  ],
  "meta": { "page": 1, "page_size": 20, "total": 3 }
```

第四章　表设计

1. 分析任务记录表

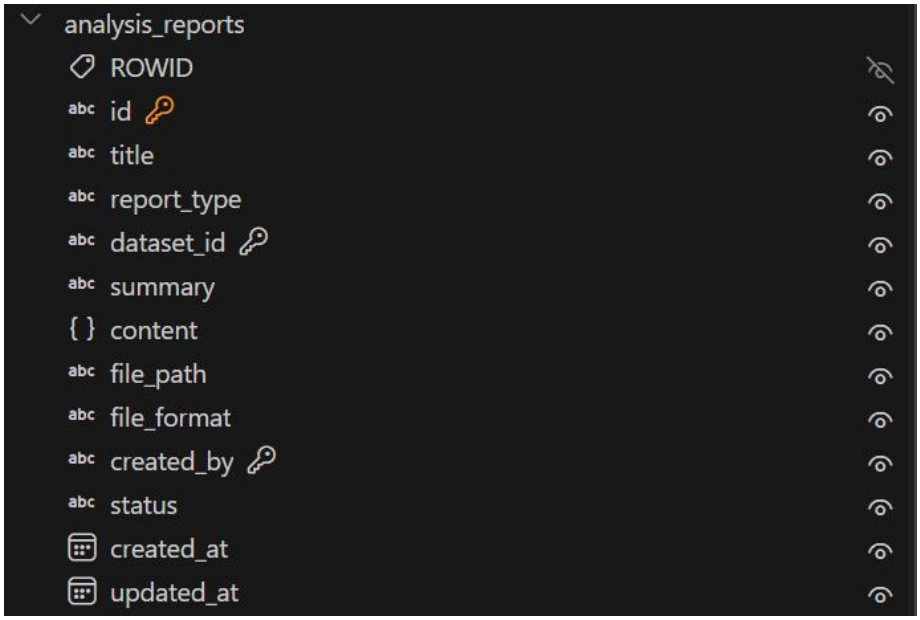
表名:analysis_jobs



备注：无

2. 分析报告信息表

表名:analysis_reports



备注：无

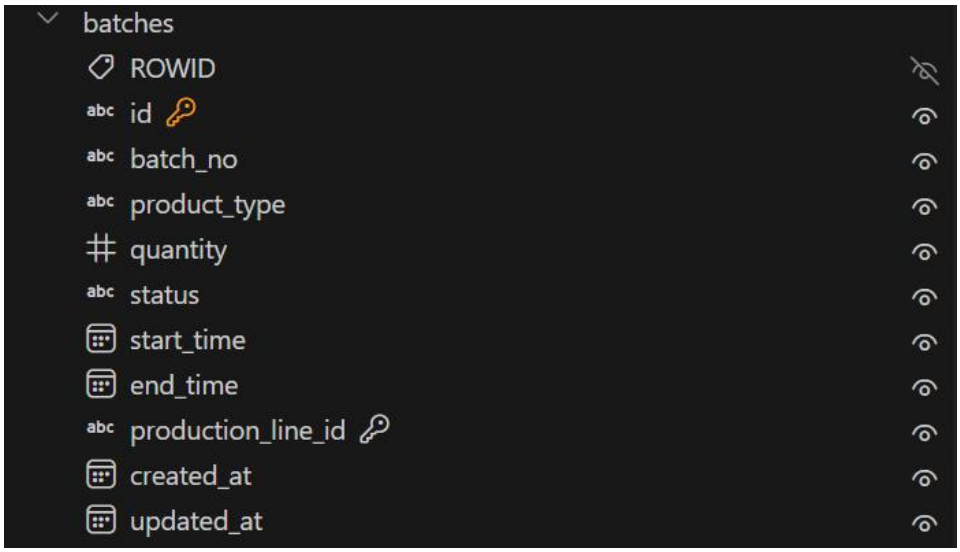
3. 系统操作审计日志表

表名:audit_logs

备注：无

4. 生产批次信息管理表

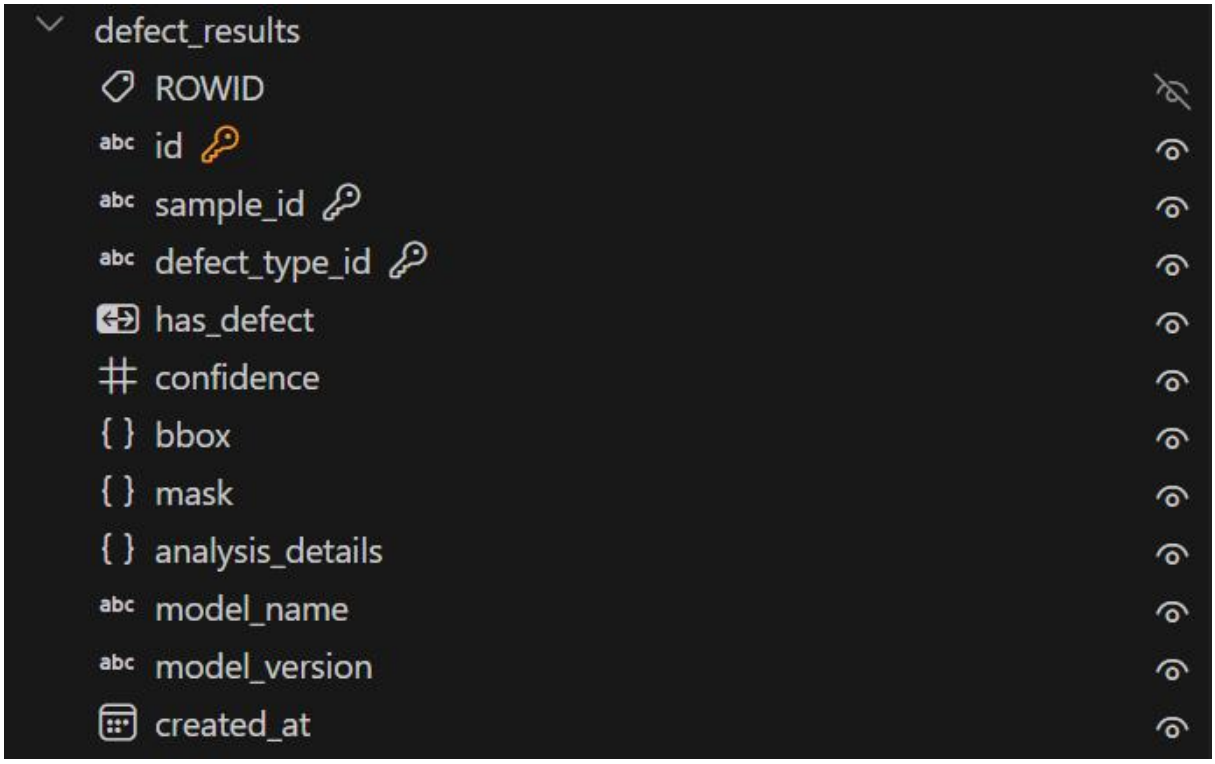
表名:batches



备注：无

5. 缺陷检测结果记录表

表名:defect_results



备注：无

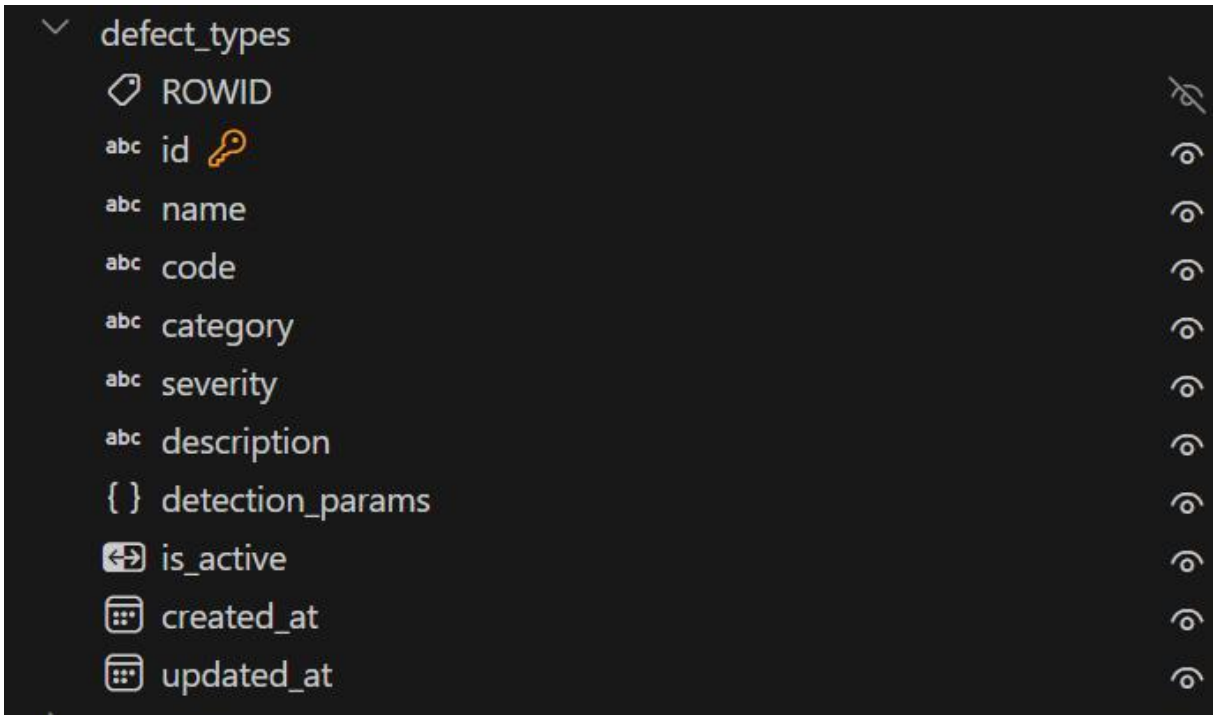
6. 缺陷样本数据管理表

表名:defect_samples

备注：无

7. 缺陷类型定义配置表

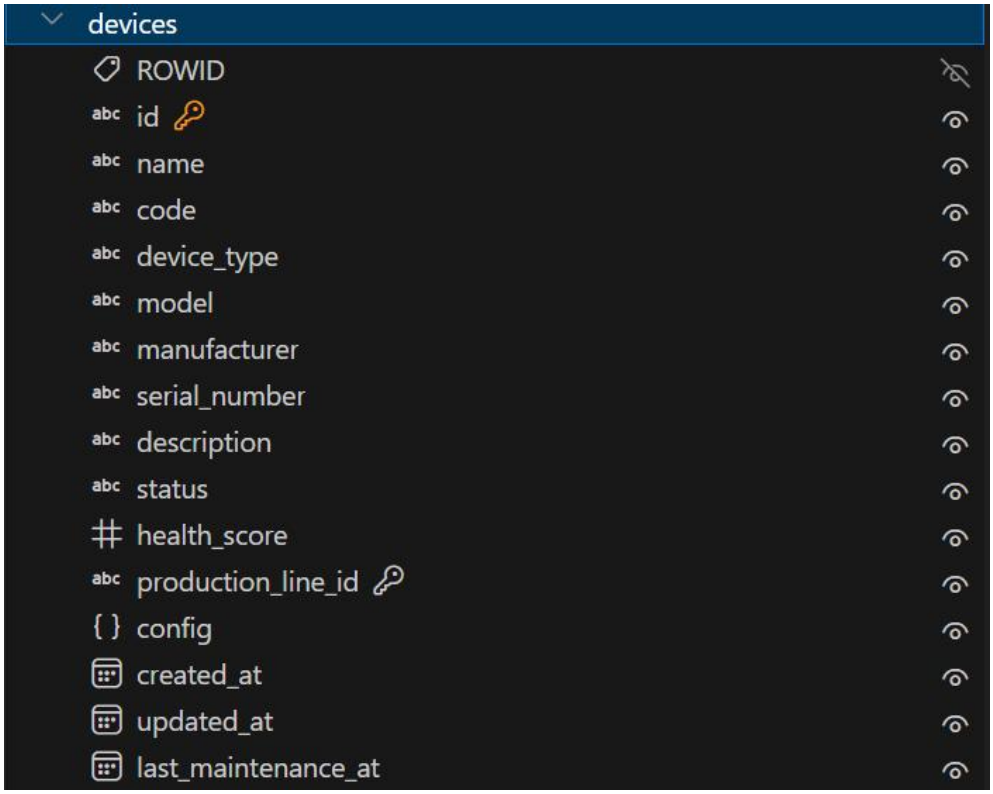
表名:defect_types



备注：无

8. 设备信息管理表

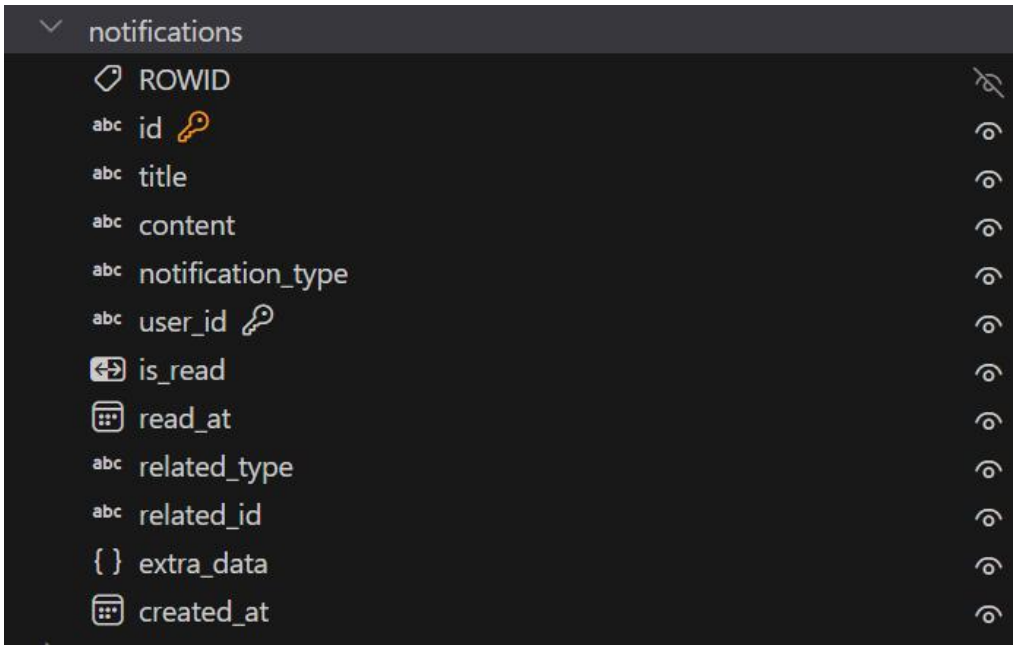
表名:devices



备注：无

9. 系统消息通知记录表

表名:notifications



备注：无

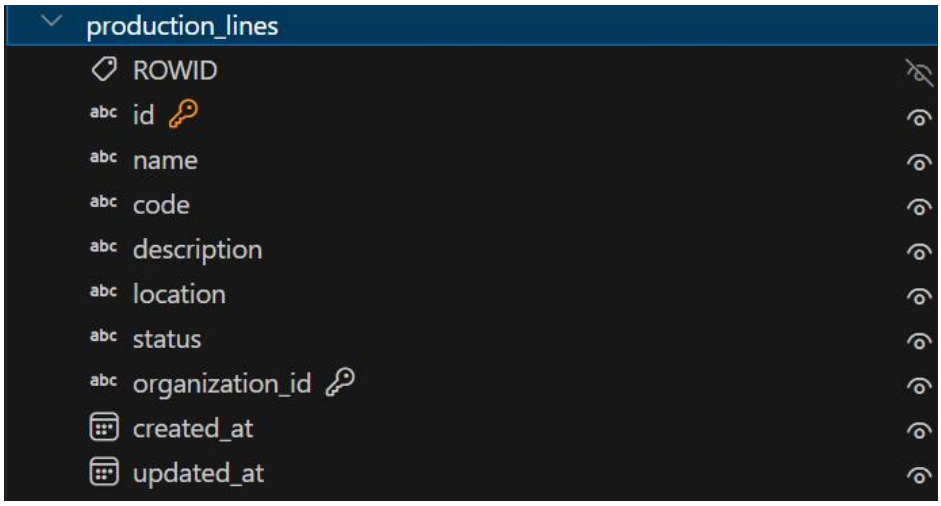
10. 组织机构信息管理表

表名:organizations

备注：无

11. 生产线信息管理表

表名:production_lines



备注：无

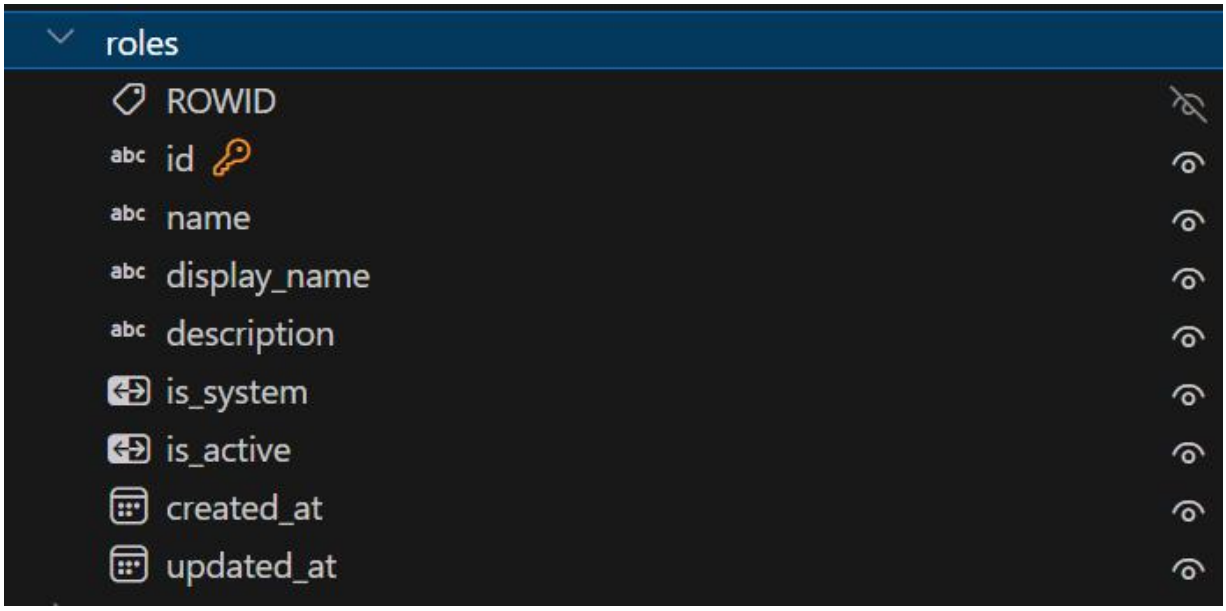
12. 角色权限关联表

表名:role_permissions

备注：无

13. 系统角色信息表

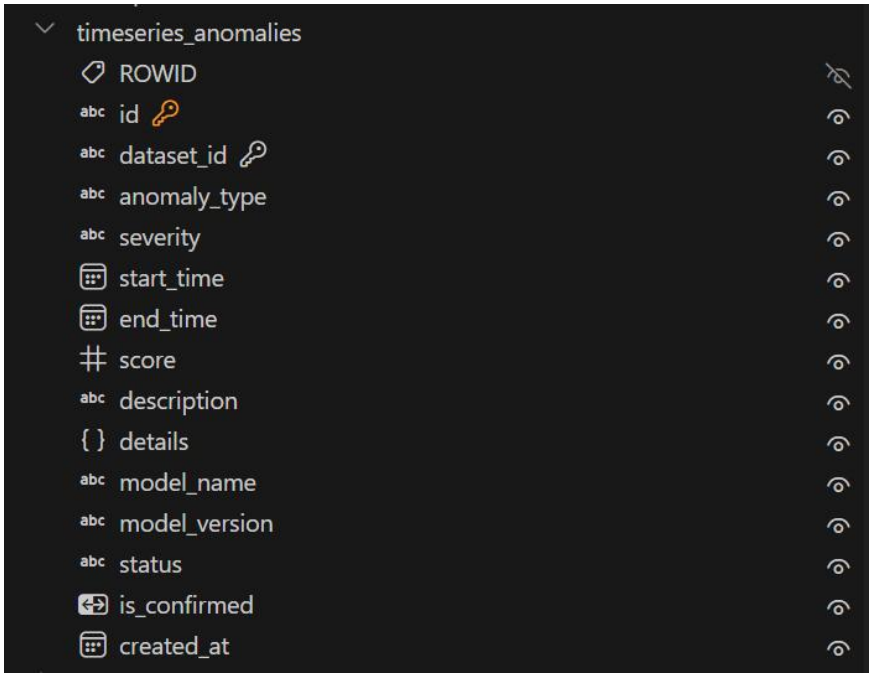
表名:roles



备注：无

14. 时序异常检测结果表

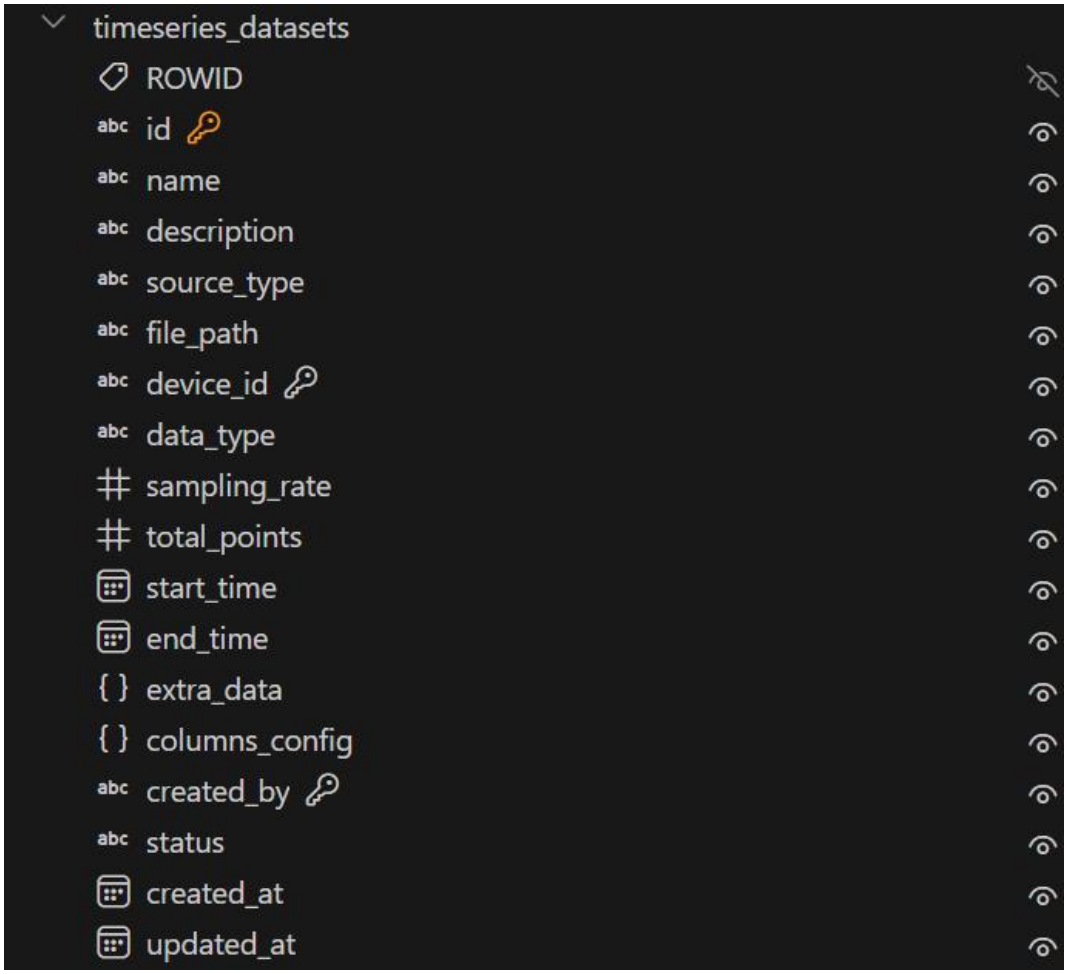
表名:timeseries_anomalies



备注：无

15. 时序数据集信息表

表名:timeseries_datasets



备注：无

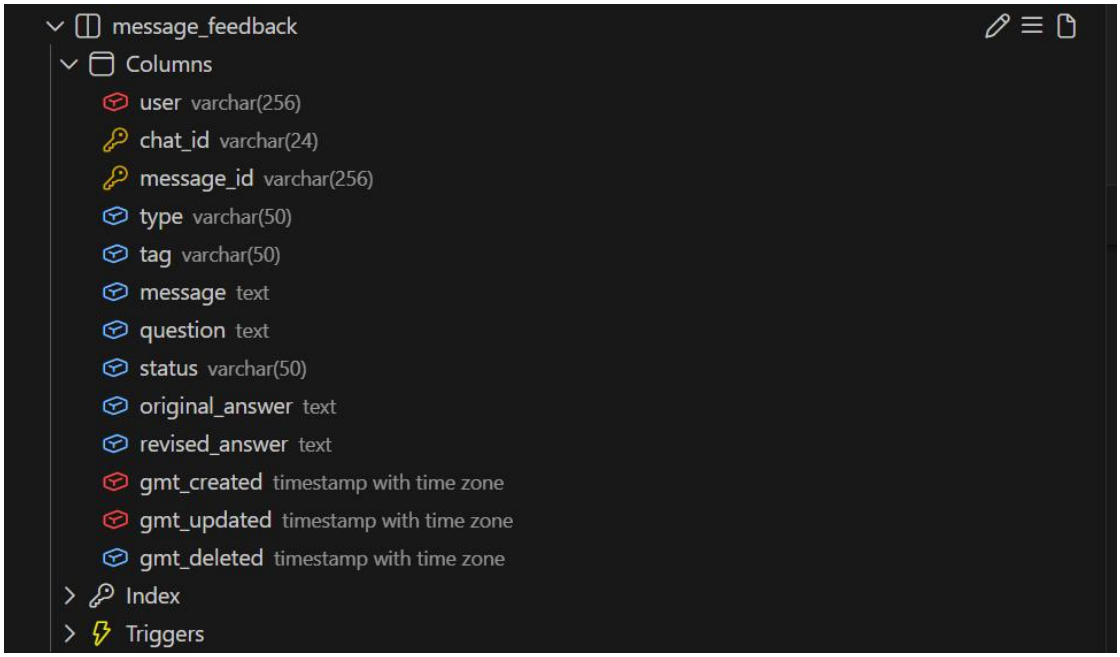
16. 时序数据点存储表

表名:timeseries_points

备注：无

17. 用户消息反馈记录表

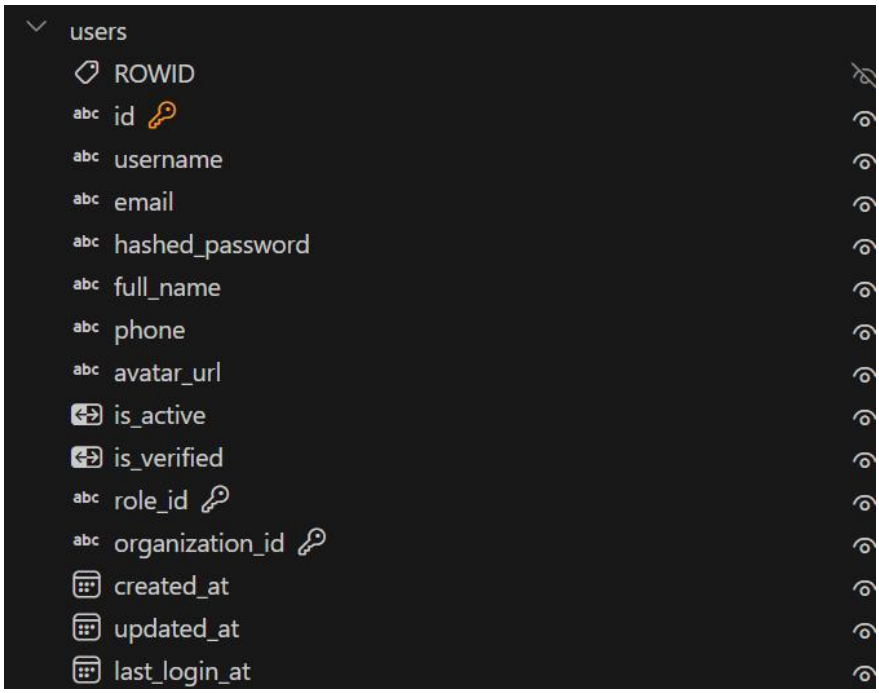
表名:message_feedback



备注：无

18. 用户表

表名:users



备注：无

第五章　非功能性设计

1. 性能设计

系统架构：后端采用“FastAPI API 服务 + Celery 异步任务 + Redis 队列 + PostgreSQL 数据库”的分层架构；检测/分析等重任务下沉至 Celery Worker/Beat，API 与长耗时处理解耦，可按服务独立扩容。

异步 API：HTTP 接口基于 FastAPI/asyncio；文件上传、列表查询等 I/O 密集环节尽量异步实现，降低阻塞，提高并发吞吐。

存储分工：PostgreSQL 负责业务与元数据（样本、结果、设备、批次、时序数据元信息）；Redis 作为缓存与消息队列；文件与模型输出挂载到 ./storage 卷，避免数据库膨胀。

任务与批处理：样本上传、时序分析、缺陷检测通过 Celery 队列异步执行，可配置并发度；建议对大批量样本/数据集分批导入、控制单批大小与队列长度，防止尖峰阻塞。

扩展能力：在 Docker Compose 中，API、Worker、Beat、Redis、Postgres 均为独立服务，可按负载水平横向扩容；数据库/Redis 连接池参数和 Uvicorn workers 数可按宿主资源调优。

2. 安全性设计

访问控制：内建用户体系，JWT 鉴权；关键接口（如管理类增删改、任务触发）通过依赖 get_current_user/require_admin 做角色权限校验。

凭证与密钥：后端统一管理 SECRET_KEY、数据库与 Redis 连接串；JWT 访问/刷新令牌分离，避免长期凭证泄露风险。

审计与追踪：AuditService 记录关键操作（用户、资源、动作、状态），便于事后审计与问题追踪。

配额与限流：可通过角色与业务逻辑限制敏感操作频次（如批量导入、检测触发），并推荐在网关/反向代理层增加请求限流与 IP 白名单。

网络与边界：建议仅暴露 80（前端）/8000（后端 API）端口；PostgreSQL、Redis 等内部服务保持容器网络访问，不直接对外开放。

3. 其他非功能性设计

易用性与界面：前端 React + Ant Design，提供仪表盘、缺陷检测、时序分析、设备/批次管理、通知中心等模块化导航；常用操作（上传样本、触发检测/分析、查看结果）在页面有明确入口与表单校验。

兼容性：后端容器化，推荐运行于主流 Linux（如 Ubuntu 22.04/24.04）；开发可在 Windows/macOS 通过 Docker 或本机 Python 3.11 环境运行。前端面向现代浏览器（Chromium/Edge/Firefox），适配 1080p 及以上分辨率。

版本与升级：数据库迁移使用 Alembic 管理，升级时执行迁移脚本保持表结构平滑演进；依赖由 requirements.txt / package.json 明确可重复安装。

可移植性与可维护性：配置集中在 .env 与 app/core/config.py，可快速切换开发/测试/生产；标准 Dockerfile 与 docker-compose.yml 一键拉起 Postgres、Redis、后端、前端、Celery Worker/Beat。后端按领域分层（api/core/models/services），Pydantic 模型与自动生成的 OpenAPI 便于协作与扩展。

观测性：内置健康检查 /api/v1/health 与结构化日志（structlog）；建议结合部署环境接入反向代理访问日志与系统级监控，满足排障与容量规划需求。